國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab04

姓名： 紀宥丞

學號： 114310221

Lab04-Ex1.
Convert the code to the Call by value
Code

```
#include <iostream>
#include <string>
using namespace std;

struct Student { //建立一個名為 Student 的結構型態
	int id;
	string name;
	int chinese;
	int english;
	int math;
	int social;
	int science;
};

// 1. 改為 Call by Address
// 參數改為 Student 結構的指標
void updateMath(Student *s) {
	// 2. 透過指標存取學生的資料，必須使用 -> 運算子
	s->math = 100;
}

// 同樣改為傳址呼叫來印出資料
void printStudent(Student *s) {
	cout << "學生編號: " << s->id << endl;
	cout << "姓名: " << s->name << endl;
	cout << "數學成績: " << s->math << endl;
}

int main() {
	Student one;
	//對新宣告的結構變數 one 中的元素賦值
	one.id = 1;
	one.name = "張小美";
	one.chinese = 77;
```

```
    one.english = 83;
    one.math = 86;
    one.social = 68;
    one.science = 91;

    // 3. 呼叫函式時，傳入變數 one 的「記憶體位址」
    // 這樣函式裡面的指標 s 就會指向 main 裡面的 one
    updateMath(&one);

    // 傳入記憶體位址來印出資料
    printStudent(&one);

    return 0;
}
```

Discussion Section

(.)：用於**直接存取**。當你手上直接擁有那個結構變數（或是它的參考 &）時使用。
(->)：用於**間接存取**。當你手上只有指向那個結構的指標（記憶體位址）時使用。
因為你只有位址，程式必須先「順著指標找到實體」，然後才能存取裡面的資料。寫 p->math 其實只是為了方便，它在底層等同於先解指標再用點運算子：(*p).math。